

1. Свойства информации: Информация обладает тремя ключевыми свойствами. **Ценность** определяется степенью ее полезности для владельца; на основе этого свойства устанавливается режим доступа (например, информация ограниченного доступа). **Достоверность** — это свойство информации с достаточной точностью отражать реальные процессы; умышленно искаженная информация называется дезинформацией. **Своевременность** — это соответствие информации определенному временному периоду, так как информация, актуальная в один момент, может устареть и потерять ценность в другой.

2. Основные понятия ЗИ в ИБС: Защита информации в информационно-вычислительных системах (ЗИ в ИБС) опирается на два основных понятия. **Информационная безопасность (ИБ)** — это состояние защищенности информации и инфраструктуры от любых воздействий, которые могут нанести ущерб. Согласно стандарту **ГОСТ Р 50.1.053-2005**, ИБ обеспечивает конфиденциальность, целостность и доступность информации. **Защита информации (ЗИ)** — это комплекс мероприятий, направленных на достижение и поддержание информационной безопасности.

3. Основные составляющие информационной безопасности: Основу информационной безопасности образует триада **КЦД (Конфиденциальность, Целостность, Доступность)**. **Конфиденциальность** — это характеристика, указывающая на необходимость введения ограничений на круг субъектов, имеющих доступ к информации. **Целостность** — свойство информации существовать в неискаженном виде. **Доступность** — свойство системы обеспечивать своевременный и беспрепятственный доступ пользователей к информации. Вся деятельность по защите информации сводится к обеспечению этих трех свойств.

4. Угроза ИБ ОС, определение. Краткая характеристика: Угроза информационной безопасности операционной системы (ОС) — это возможность реализации воздействия на информацию или компоненты системы, приводящего к ее искажению, уничтожению, копированию, блокированию доступа или сбою функционирования. Попытка реализации угрозы называется **атакой**, а лицо, ее предпринимающее — **злоумышленником**. Угрозы часто являются следствием **уязвимых мест** в защите, а период времени, когда уязвимость может быть использована, называется **"окном опасности"**.

5. Классификация угроз ОС: Угрозы информационной безопасности классифицируются по нескольким основаниям. **По аспекту ИБ**, против которого они направлены: угрозы конфиденциальности, целостности и доступности. **По компонентам информационной системы (ИС)**, на которые нацелены: данные, программы, аппаратное обеспечение, поддерживающая инфраструктура. **По способу осуществления:** случайные, преднамеренные, природного или техногенного характера. **По расположению источника угроз:** изнутри системы и извне.

6. Угрозы конфиденциальности: Эти угрозы направлены на несанкционированное получение доступа к защищаемой информации. Они делятся на два типа: **раскрытие служебной информации** (хищение паролей, ключей шифрования) и **раскрытие предметной информации** (утечка коммерческой или государственной тайны). Конкретные способы реализации включают **перехват данных** при передаче, **хищение носителей информации, маскрад** (действие под видом легитимного пользователя) и **злоупотребление полномочиями**.

7. Угрозы целостности: Угрозы целостности нацелены на несанкционированное изменение или уничтожение информации. Они подразделяются на угрозы **статической целостности** (направлены на сами данные и программы: ввод неверных данных, модификация, уничтожение, внедрение вредоносного ПО) и **динамической целостности** (нарушение корректности процессов: нарушение атомарности транзакций, переупорядочение или дублирование сообщений, отказ от совершаемых действий). Важным источником таких угроз являются внутренние злоумышленники.

8. Угрозы доступности: Угрозы доступности создают условия, при которых доступ к информации или службе для легитимных пользователей становится невозможным или затрудненным. Классифицируются по компонентам ИС: **отказ пользователей** (нежелание или невозможность работы с системой), **внутренний отказ ИС** (ошибки, сбои ПО и аппаратуры, саботаж) и **отказ поддерживающей инфраструктуры** (нарушение электропитания, связи, стихийные бедствия). Особо опасны атаки на доступность, такие как **агрессивное потребление ресурсов** и **скоординированные распределенные атаки (DDoS)**.

9. Политика безопасности (уровень организации): Это политика верхнего уровня, представляющая собой совокупность документированных решений руководства организации. Она задает общее направление, формулируя цели в области ИБ. Политика определяет структуру и ответственность, обеспечивает базу для законопослушности, управляет ресурсами и координирует взаимодействие. Ее приоритеты зависят от специфики организации (например, для режимного предприятия — конфиденциальность, для интернет-магазина — доступность).

10. Политика безопасности (уровень отделов/служб): Это политика среднего уровня, которая детализирует общие положения верхнеуровневой политики применительно к конкретным аспектам безопасности. Примеры таких политик: политика использования паролей, политика доступа в интернет. Ее структура для каждого аспекта должна освещать: описание аспекта, область применения, позицию организации, роли и обязанности, а также контакты для разъяснений.

11. Политика безопасности (уровень информационных сервисов): Это политика нижнего уровня, относящаяся к конкретным информационным сервисам (например, сервер электронной почты, файловое хранилище). В отличие от верхних уровней, она должна быть максимально подробной и конкретной. Документ четко отвечает на ключевые вопросы: кто имеет право доступа, к каким именно объектам, при каких условиях доступ предоставляется и как организован удаленный доступ. Правила безопасности формализуются для простоты автоматизации.

12. Программа безопасности — это практический план реализации положений политики безопасности. Если политика отвечает на вопросы "что" и "зачем" защищать, то программа — "как", "кем" и "когда" это будет сделано. Она структурируется на два уровня: **программа верхнего уровня** (центральная, охватывает всю организацию,

управление рисками, стратегическое планирование) и **программа нижнего уровня** (служебная, обеспечивает защиту конкретных сервисов, выбор и установку технических средств).

13. Синхронизация программы безопасности с этапами жизненного цикла ИС: Для максимального эффекта программа безопасности должна быть тесно связана с жизненным циклом защищаемого информационного сервиса. На этапе **инициации** проводится оценка критичности сервиса и информации. На этапе **закупки** формулируются окончательные требования к защитным средствам. При **установке** выполняется конфигурация встроенных средств безопасности и разработка регламентов. В процессе **эксплуатации** проводятся периодические проверки для противодействия накоплению изменений, снижающих безопасность. При **выводе из эксплуатации** осуществляется безопасный перенос, архивирование или уничтожение данных.

14. Дискреционная политика безопасности (Discretionary Access Control - DAC) основана на управлении доступом, при котором права субъектов к объектам определяются заданными правилами, а владелец объекта или администратор может по своему усмотрению (дискреции) изменять эти права для других. Классическим механизмом реализации является **матрица доступов**, которая на практике в ОС реализуется в виде **списков управления доступом (ACL - Access Control List)**. Ее достоинство — относительная простота реализации и понятность. Главный недостаток — слабая защита от "тройского коня", так как пользователь, имеющий право на чтение информации, может скопировать ее в общедоступное место.

15. Политика ролевого разграничения доступа (Role-Based Access Control - RBAC) является развитием дискреционной политики. Права доступа назначаются не напрямую пользователям, а **ролям** (например, "бухгалтер", "менеджер"), которым затем назначаются пользователи. Это упрощает администрирование, так как добавление нового пользователя сводится к назначению ему роли. RBAC позволяет легче реализовывать **принцип минимальных привилегий**, делает правила доступа более прозрачными и способствует соблюдению регуляторных требований.

16. Мандатная политика безопасности (Mandatory Access Control - MAC), или полномочная, устанавливается централизованно администрацией системы и не может быть изменена пользователями. Ее основой являются **метки безопасности** (уровни конфиденциальности), присваиваемые всем субъектам и объектам. Основная цель — предотвращение утечки информации от объектов с высоким уровнем конфиденциальности к объектам с низким уровнем. В отличие от дискреционной политики, пользователь, даже будучи владельцем файла, не может скопировать его в файл с низким грифом, что защищает от злонамеренных действий самого пользователя.

17. Модель Белла — ЛаПадулы — это классическая модель мандатного управления доступом, разработанная для защиты **конфиденциальности**. Ее ключевые правила безопасности: **"no read up"** — субъект не может читать объект с более высоким уровнем секретности. **"no write down"** — субъект не может записывать в объект с более низким уровнем секретности (это предотвращает утечку). Недостатки модели включают возможность **деклассификации** (обход правил через понижение уровня) и проблемы в распределенных системах.

18. Модель Бибба является прямой противоположностью модели Белла-ЛаПадулы и защищает **целостность** информации. Ее основные правила: **"no read down"** — субъект не может читать объект с более низким уровнем целостности (чтобы не "загрязнить" себя); **"no write up"** — субъект не может записывать в объект с более высоким уровнем целостности (чтобы ненадежный субъект не испортил надежные данные). Модель наследует некоторые недостатки Белла-ЛаПадулы и сильно полагается на доверенные процессы.

19. Средства безопасности, предоставляемые ОС Windows: ОС Windows предоставляет комплекс встроенных средств безопасности. **Безопасный вход в систему (аутентификация)** обеспечивается процессами Winlogon.exe и Lsass.exe. **Управление доступом** реализует дискреционную модель на основе списков управления доступом (ACL), ядром механизма является **Монитор контроля безопасности (Security Reference Monitor, SRM)**. **Аудит безопасности** регистрирует события. **Защита от повторного использования объекта** предотвращает утечку данных через освобожденную память. **Мандатный контроль целостности (Windows Integrity Control - WIC)** предотвращает взаимодействие ненадежных процессов с системными объектами.

20. Идентификация пользователей в ОС Windows: основана на **идентификаторах безопасности (Security Identifiers, SID)**. Это уникальные числовые значения переменной длины. После успешной аутентификации для пользователя создается **маркер доступа** — объект ядра, который сопровождает каждый процесс пользователя и содержит SID пользователя, SID его групп, список привилегий и уровень целостности.

21. Защита объектов в ОС Windows: Защита объектов основана на дискреционной модели доступа и реализуется с помощью **маркера доступа** (у субъекта) и **дескриптора безопасности** (у объекта). Дескриптор безопасности содержит SID владельца, **дискреционный список управления доступом (DACL)**, который определяет разрешения и запреты, и **системный список управления доступом (SACL)** для настройки аудита. При проверке доступа **Запрещающие ACE (элементы управления доступом) всегда имеют приоритет** над разрешающими.

22. Реализация мандатного механизма доступа в ОС Windows: Мандатный механизм в Windows реализован как **Windows Integrity Control (WIC)**. Всем субъектам (процессам) и объектам присваиваются **уровни целостности** (от "Незаслуживающий доверия" до "Системный"). Ключевое правило: процесс с низким уровнем целостности не может получить доступ к объекту с высоким уровнем целостности, даже если DACL это разрешает. Это предотвращает повреждение системных файлов вредоносным ПО. **Контроль целостности WIC имеет более высокий приоритет, чем проверка дискреционных прав в DACL.**

23. Разграничение доступа к файлам в ОС Linux: Дискреционное управление доступом в Linux базируется на модели владения. Права определяются для трех категорий: **Владелец (Owner), Группа (Group) и Остальные (Others)**. Базовые права — **Read (чтение), Write (запись), Execute (выполнение)**. Также существуют особые атрибуты: **SUID** (файл выполняется с правами владельца), **SGID** (файл выполняется с правами группы / наследование группы для каталогов) и **Sticky-bit** (в каталоге файлы может удалить только их владелец). Для изменения прав используются утилиты **chmod, chown, chgrp**.

24. Управление файлами ОС Linux: Логическая структура файловой системы Linux — единое дерево каталогов с корнем /. Существуют два типа ссылок. **Жесткая ссылка** — дополнительное имя для существующего файла (тот же inode). **Символическая ссылка** — специальный файл-указатель, содержащий путь к целевому файлу (имеет свой inode). Основные команды для управления: **ls** (список файлов), **cd** (смена каталога), **cp, mv, rm** (копирование, перемещение, удаление). Для навигации и управления можно использовать текстовый файловый менеджер **Midnight Commander (mc)**.

25. Проблемы реализации мандатного управления доступом в ОС: Нарушитель может обойти контроль через **неконтролируемые сущности** (объекты без мандатной метки). Возможны **несанкционированные информационные потоки по памяти и создание каналов утечки по времени**. Наличие **специальных привилегий** позволяет обходить правила МУД. Политики безопасности сложны для описания и требуют тонкой настройки (**сложность администрирования**). Большинство прикладного программного обеспечения требует особой настройки для работы в среде с МУД (**проблемы совместимости**).

26. Реализация мандатного управления доступом в ОСCH Astra Linux: В операционной системе специального назначения (ОСCH) Astra Linux МУД реализовано с помощью собственной подсистемы безопасности **PARSEC** и основывается на двух типах метод: конфиденциальности и целостности. Эта реализация повышает защищенность, так как преднамеренное преодоление МУД требует значительных усилий. Будучи собственной разработкой, PARSEC лучше контролируется и верифицируется. Действия по обходу МУД выявляются развитой системой аудита. Администрирование осуществляется через графическую утилиту "Управление политикой безопасности".

27. Мандатный контроль целостности в ОСCH Astra Linux реализован как часть подсистемы PARSEC для защиты от модификации критически важных компонентов ОС. Он использует уровни целостности ("Низкий" и "Высокий"). Ключевое правило: процесс может модифицировать объект только если его уровень целостности **не ниже** уровня целостности объекта. Это правило действует независимо от других прав доступа. Используется для помечания процессов (системный/несистемный).

28. Управление доступом к объектам графической подсистемы в ОСCH Astra Linux: Для защиты от уязвимостей X Window System в Astra Linux используется **механизм изоляции**. Процессы с мандатными атрибутами, отличными от стандартных, запускаются в изолированных графических сессиях с использованием виртуального X-сервера. Вывод изолированного приложения проецируется в окно основной сессии как монолитный графический образ, чья внутренняя структура недоступна для анализа или воздействия. Визуальные признаки изолированного окна: пометка в заголовке, цветная рамка и изолированный буфер обмена. Сюда также можно отнести графические киоски.

29. Администрирование мандатного управления доступом в ОСCH Astra Linux: Основное средство администрирования — графическая утилита **"Управление политикой безопасности"**. С ее помощью настраиваются уровни конфиденциальности, неиерархические категории и уровни целостности, которые затем назначаются пользователям. Для работы с мандатными атрибутами из командной строки используются утилиты: **rdp-ulbls** (просмотр и изменение атрибутов пользователей), **getfmas** (получение атрибутов файла), **rdpl-file** (изменение атрибутов файла). Для администрирования МУД существуют **специальные привилегии (capabilities)**, например, **parsec_cap_chmas** для изменения мандатных атрибутов сущностей.

30. Организация аутентификации: Аутентификация — это процесс проверки подлинности субъекта (пользователя), подтверждающий, что он является тем, за кого себя выдает. В основе лежит использование факторов: **знание (пароль), обладание (токен) и свойство** (биометрия). В ОС семейства Unix/Linux используется модульная архитектура **PAM (Pluggable Authentication Modules)**, которая позволяет гибко настраивать методы аутентификации для разных сервисов. В защищенных ОС, таких как Astra Linux, процесс аутентификации тесно интегрирован с назначением мандатных атрибутов для сеанса пользователя.

31. Основные атаки на протоколы аутентификации: К основным атакам относятся: **Маскарад** — выдача себя за легитимного пользователя. **Повторная передача (Replay Attack)** — перехват и повтор данных успешного сеанса. **Подмена стороны аутентификационного обмена (Interleaving Attack)** — активное вмешательство в процесс обмена. **Принудительная задержка** — намеренная задержка передаваемых сообщений. **Атака с выборкой текста** —

попытка получить информацию о ключах, анализируя ответы системы. Для противодействия используются механизмы "запрос-ответ", нонсы, метки времени и цифровые подписи.

32. Методы аутентификации, использующие пароли и PIN-коды: Эти методы относятся к простой аутентификации по фактору **"знание"**. **Пароль** — это секретная многобуквенная последовательность символов. Для защиты пароли должны храниться не в открытом виде, а в виде **образа пароля** — результата применения криптостойкой хэш-функции с "солью". **PIN-код** — это короткий, обычно цифровой код, используемый для аутентификации держателя устройства (смарт-карты, токена). Основные недостатки методов: уязвимость к перехвату, подбору, фишингу и повторной передаче.

33. Аутентификация на основе одноразовых паролей: Это метод, при котором для каждого сеанса используется новый, уникальный пароль, что обеспечивает защиту от атак с повторной передачей. Пароль динамически меняется на основе постоянного секрета и изменяющегося параметра. Пример реализации — **аппаратный токен с временной синхронизацией (TOTP)**. Токен и сервер аутентификации синхронизированы по времени, и токен с заданным интервалом генерирует новое число (пароль), которое пользователь вводит при входе. Сервер, зная секретный ключ и время, выполняет тот же расчет для проверки.

34. Строгая аутентификация, основанная на симметричных алгоритмах: В этом методе проверяющая и доказывающая стороны заранее разделяют один и тот же **секретный ключ**. Доказывающая сторона подтверждает подлинность, демонстрируя знание ключа, не раскрывая его. Классические протоколы используют шифрование случайного числа (**нонса**), полученного от проверяющей стороны. Пример: сторона А отправляет нонс r_B , сторона А возвращает $E_K(r_B, B)$. В расшифровывает сообщение и проверяет, что вернулось его случайное число. В больших системах используется доверенный сервер аутентификации (например, Kerberos).

35. Строгая аутентификация, основанная на асимметричных алгоритмах: В этих протоколах используется **криптография с открытым ключом**. Доказывающая сторона подтверждает подлинность, демонстрируя знание своего **секретного ключа**. Это можно сделать двумя способами: 1) **Расшифровка запроса**, зашифрованного на открытом ключе доказывающей стороны. 2) **Создание цифровой подписи** на запросе от проверяющей стороны с использованием секретного ключа. **Пример 1 (расшифровка):** *Сервер → Клиент:* Зашифрованное случайное число открытым ключом клиента; *Клиент → Сервер:* Расшифрованное число; *Проверка:* Если числа совпали — клиент владеет секретным ключом. **Пример 2 (подпись):** *Сервер → Клиент:* Случайное число; *Клиент → Сервер:* Цифровая подпись этого числа; *Проверка:* Сервер проверяет подпись открытым ключом клиента.

36. Аутентификация, основанная на использовании цифровой подписи: Это частный случай строгой аутентификации на асимметричных алгоритмах. Доказывающая сторона подписывает своим **секретным ключом** набор данных (нонс, метку времени, идентификаторы). Проверяющая сторона верифицирует подпись с помощью **открытого ключа** доказывающей стороны. Этот метод не только обеспечивает **аутентификацию**, но и гарантирует **целостность** переданных данных и **неотказуемость** (отправитель не может отказаться от факта отправки). Для доверия открытому ключу используются **цифровые сертификаты X.509**.

37. Активный аудит — это оперативный анализ накопленной информации (журналов событий) в реальном времени с автоматическим реагированием на подозрительные ситуации. Он направлен на выявление двух типов активности: **атаки** (попытки незаконного получения полномочий) и **злоупотребление полномочиями** (действия в рамках прав, но нарушающие политику). Основные методы обнаружения: **Сигнатурный анализ** (сравнение с известными шаблонами атак) и **Статистический анализ (аномалий)** (выявление отклонений от нормального поведения).

38. Контроль целостности сообщений обеспечивает уверенность в том, что данные не были изменены при передаче. Для этого используется **хэш-функция**. Отправитель вычисляет для сообщения M короткое **хэш-значение (дайджест)** $m = h(M)$ и передает его вместе с сообщением. Получатель самостоятельно вычисляет $m' = h(M)$ и сравнивает с полученным m . Совпадение подтверждает целостность. Для защиты от подмены дайджеста используется **ключевая хэш-функция (HMAC)**, где дайджест вычисляется с использованием секретного ключа K : $m = h(M, K)$. **Электронная цифровая подпись (ЭЦП)** является более мощным средством, обеспечивающим одновременно целостность, аутентификацию источника и неотказуемость.

39. Особенности аутентификации в ОСCH Astra Linux: Аутентификация в Astra Linux построена на стандартной архитектуре **PAM**, но имеет важные расширения. К стандартному набору PAM добавлены **специализированные модули**, которые назначают мандатные атрибуты и уровень целостности первому процессу в сеансе пользователя. Если программа в ходе аутентификации не обращается к этим модулям, сеанс получает низкий уровень целостности, что блокирует доступ к конфиденциальным данным. Также действует **строгая политика паролей**, администрируемая через утилиту **fly-admin-smc**, которая предписывает минимальную длину, использование символов разных классов, максимальный срок действия и автоматическую блокировку учетной записи при неудачных попытках входа.

40. Особенности аудита в ОСCH Astra Linux: Система аудита в Astra Linux является **двухурневой**. Первый уровень — стандартный аудит, обеспечиваемый демоном **rsyslogd**. Второй уровень — **мощная собственная подсистема аудита PARSEC**, специально разработанная для регистрации событий безопасности. Она записывает детальную информацию о событиях ядра и пользователя в файлы **/var/log/parsec/kernel.mlog** и **user.mlog**. Для просмотра и анализа используется графическая утилита **fly-admin-viewaudit** с мощными возможностями фильтрации (по времени, статусу, типу системного вызова и др.). Политика аудита гибко настраивается через утилиту **fly-admin-smc** с использованием битовых масок для успешных и неуспешных событий.